UNIT-I – Introduction to .Net Framework

.NET : .NET is a free, cross-platform, open source developer platform for building many different types of applications. With .NET, you can use multiple languages, editors and libraries to build for web, mobile, desktop, games and IoT.

Key aspects of .NET:

- **Cross-platform:** .NET code can be executed on different operating systems, not just Windows.
- **Open-source:** The .NET platform is open-source, meaning its code is publicly available and can be modified and distributed by anyone.
- **Multi-language support:** While C# is a prominent language for .NET, it also supports other languages like F# and VB.NET.
- **Rich libraries and tools:** .NET provides a vast set of libraries and tools that simplify common development tasks, such as working with databases, handling network requests, or creating user interfaces.
- **Managed execution:** .NET applications run within a managed environment called the Common Language Runtime (CLR), which handles tasks like memory management, security, and exception handling.

Benefits of using .NET:

- **Increased productivity:** The .NET platform provides a consistent programming model and a rich set of libraries, which can significantly speed up development.
- **Improved performance:** .NET is known for its performance capabilities, especially when working with high-scale applications.
- **Enhanced security:** .NET offers features like automatic memory management and type safety, which contribute to more secure applications.
- **Simplified deployment:** .NET's cross-platform capabilities make it easier to deploy applications to different environments.

Components (.NET includes the following components:)

- **Runtime** -- executes application code.
- **Libraries** -- provides utility functionality like JSON parsing.

- **Compiler** -- compiles C# (and other languages) source code into (runtime) executable code.
- **SDK and other tools** -- enable building and monitoring apps with modern workflows.
- **App stacks** -- like ASP.NET Core and Windows Forms, that enable writing apps.

Key features of .NET

.NET includes several key features that make it a powerful development platform:

- **Common Language Runtime (CLR):** The runtime environment that manages memory, handles exceptions, and provides security for your applications.
- **Base Class Library (BCL):** A comprehensive collection of reusable types for common tasks like file handling, database connection, and network communication.
- **Language Interoperability:** Code written in one .NET language can work seamlessly with code written in another .NET language.
- **Garbage Collection:** Automatic memory management that frees developers from manual memory allocation and deallocation.
- **Just-In-Time (JIT) Compilation**: Converts intermediate language code to machine code at runtime for optimized performance.
- **Asynchronous Programming Model:** Built-in support for writing asynchronous code that improves responsiveness and scalability.

What can you build with .NET?

.NET is incredibly versatile, enabling developers to create a wide range of applications:

- **Web Applications:** Using ASP.NET Core, you can build modern, cloud-based, internet-connected applications and APIs.
- **Desktop Applications:** Create Windows desktop applications using Windows Forms, WPF (Windows Presentation Foundation), or the newer MAUI (Multi-platform App UI).
- **Mobile Applications:** Build cross-platform mobile apps for iOS and Android using .NET MAUI or Xamarin.
- **Cloud Services:** Develop microservices, serverless functions, and cloud-native applications that can be deployed to Azure or other cloud platforms.
- **Game Development:** Create games using Unity with C# as the scripting language.
- **IoT Applications:** Build applications for Internet of Things devices, from Raspberry Pi to industrial IoT solutions.
- **Machine Learning:** Use ML.NET to add machine learning capabilities to your .NET applications without requiring expertise in data science.

- **Artificial Intelligence:** Integrate AI capabilities through services like Azure Cognitive Services or custom ML.NET models.

Difference between ASP & ASP.NET.

- ASP is mostly written using VB Script and HTML intermixed. Presentation and business logic is intermixed while ASP.NET can be written in several .NET compliant languages such as C# or VB.NET.
- ASP has maximum of 4 built in classes like Request, Response, Session and Application whereas ASP.NET using .NET framework classes which has more than 2000 built-in classes.
- ASP does not have any server based components whereas ASP.NET offers several server based components like Button, TextBox etc. and event driven processing can be done at server.
- ASP doesn't support Page level transactions whereas ASP.NET supports Page level transactions.
- ASP.NET offers web development for mobile devices which alters the content type (wml or chtml etc.) based on the device.
- ASP.NET allows separation of business and presentation logic because the code need not be included directly in the *.aspx page.
- ASP.NET uses languages which are fully object oriented languages like C# and also supports cross language support.
- ASP.NET offers support for Web Services and rich data structures like DataSet which allows disconnected data processing.

Benefits of ASP.Net over ASP Technology

ASP.Net is the next generation of Microsoft's Active Server Page (ASP) technology platform. ASP.Net is superior to ASP in the following ways:

- **Compiled Code –** ASP.Net provides greatly increased performance by running compiled versus interpreted code.
- **Language Support –** While ASP supported VBScript (a subset of Visual Basic) and Jscript (a subset of Java), ASP.Net supports standard programming languages including Visual Basic, C# and C++.
- **Strict Coding Requirements –** Developers are forced to adhere to strict coding standards. This promotes a professional engineering approach to ASP.Net application development and results in code that is generally less buggy than its ASP counterparts.
- **Event-Driven Programming Model** – All ASP.Net objects on a Web page expose events that can be processed by ASP.Net code. Handling events such as Load, Click and Change via code reduces program complexity and increases organization.

- **Third-Party Controls –** The ASP.Net community has been growing rapidly for the last two years. There is a large base of high-quality third-party controls that can be obtained to significantly speed up development time and reduce cost.
- **User Authentication –** ASP.Net supports forms-based user authentication, including cookie management and automatic redirection of unauthorized logins. ASP.Net allows for user accounts and roles thus providing a high degree of granularity for controlling access to objects and pages.

List Advantages' of ASP.NET

- ASP.NET hugely reduces the amount of code required to build large applications.
- With built-in Windows authentication and per-application configuration, your applications are safe and secured.
- It provides better performance by taking advantage of early binding, just-in-time compilation, native optimization, and caching services right out of the box.
- The ASP.NET framework is complemented by a rich toolbox and designer in the Visual Studio integrated development environment. WYSIWYG editing, drag-and-drop server controls, and automatic deployment are just a few of the features this powerful tool provides.
- Provides simplicity as ASP.NET makes it easy to perform common tasks, from simple form submission and client authentication to deployment and site configuration.
- The source code and HTML are together therefore ASP.NET pages are easy to maintain and write. Also the source code is executed on the server. This provides a lot of power and flexibility to the web pages.
- All the processes are closely monitored and managed by the ASP.NET runtime, so that if process is dead, a new process can be created in its place, which helps keep your application constantly available to handle requests.
- It is purely server-side technology so, ASP.NET code executes on the server before it is sent to the browser.
- Being language-independent, it allows you to choose the language that best applies to your application or partition your application across many languages.
- ASP.NET makes for easy deployment. There is no need to register components because the configuration information is built-in.
- The Web server continuously monitors the pages, components and applications running on it. If it notices any memory leaks, infinite loops, other illegal activities, it immediately destroys those activities and restarts itself.
- Easily works with ADO.NET using data-binding and page formatting features. It is an application which runs faster and counters large volumes of users without having performance problems

What is .Net Framework?

.Net Framework is a software development platform developed by Microsoft for building and running Windows applications. The .Net framework consists of developer tools, programming languages, and libraries to build desktop and web applications. It is also used to build websites, web services, and games.

When referring to other development environments, as in the preceding paragraph, I'm focusing on the traditional practice of compiling to an executable file that contains machine code and how that file is loaded and executed by the operating system. Figure 1.1 shows what I mean about the traditional compilation-to-execution process.

Figure 1.1 Traditional compilation.

In the traditional compilation process, the executable file is binary and can be executed by the operating system immediately. However, in the managed environment of .NET, the file produced by the compiler (the C# compiler in our case) is not an executable binary. Instead, it is an assembly, shown in Figure 1.2, which contains metadata and intermediate language code.

Figure 1.2 Managed compilation.

Evolution of .NET Technology

There are three significant phases of the development of .NET technology.

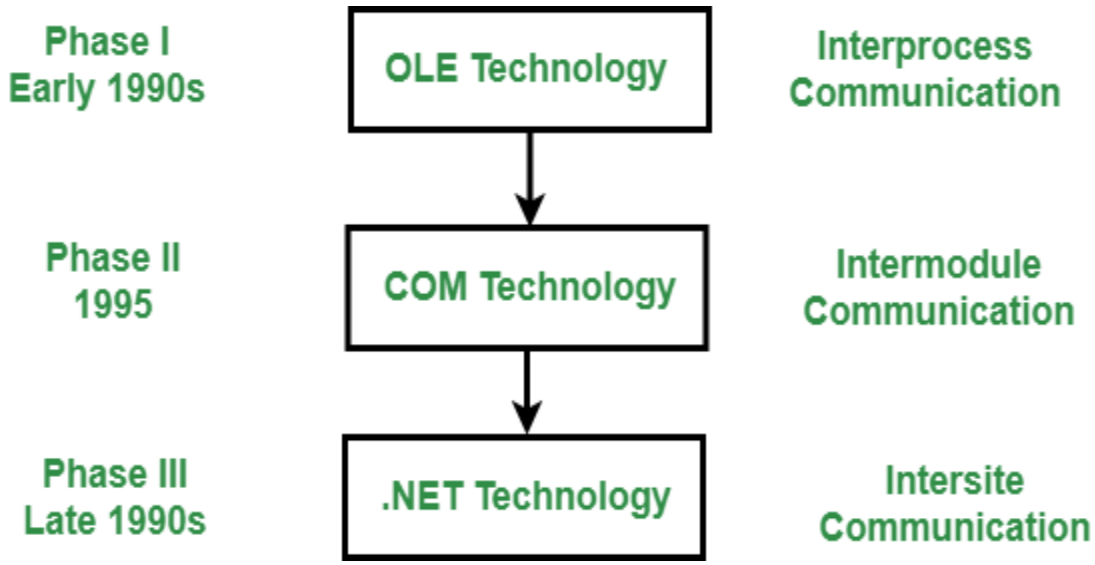
1. OLE Technology
2. COM Technology
3. .NET Technology

Evolution of .NET

OLE Technology: OLE (Object Linking and Embedding) is one of the technologies of Microsoft's component document. Basically, its main purpose is to link elements from different applications with each other.

COM Technology: The technology of the Microsoft Windows family of the operating system, Microsoft COM (Common Object Model) enables various software components to communicate. COM is mostly used by developers for various purposes like creating reusable software components, linking components together to build applications, and also taking advantage of Windows services. The objects of COM can be created with a wide range of programming languages.

.NET Technology: .NET technology of collection or set of technologies to develop windows and web applications. The technology of .Net is developed by Microsoft and was launched in Feb. 2002, by basic definition, Microsoft's new Internet Strategy. It was originally called NGWS (Next Generation Web Services). It is considered to be one of the most powerful, popular, and very useful Internet Technology available today.



Why we need .NET?

Productive: .NET helps you develop high quality applications faster. Modern language constructs like generics, Language Integrated Query (LINQ), and asynchronous programming make developers productive. Combined with the extensive class libraries, common APIs, multi-language support, and the powerful tooling provided by the Visual Studio family, .NET is the most productive platform for developers. .NET is a modern, innovative, open source development platform and developers love it.

Any app, any platform : With .NET you can target any application type running on any platform. Developers can reuse skills and code across all of them in a familiar environment. That means developers can build apps faster, with less cost. From mobile applications running on iOS, Android, and Windows, to Enterprise server applications running on Windows Server and Linux, or high-scale microservices running in the cloud, .NET provides a solution for you.

Performance where it matters: .NET is fast. Really fast! That means applications provide better response times and require less compute power. The popular TechEmpower benchmark compares web application frameworks with tasks like JSON serialization, database access, and server side template rendering - .NET performs faster than any other popular framework.

Trusted and secure : The .NET platform is officially supported by Microsoft and trusted by thousands of companies and millions of developers. Microsoft takes security very seriously and releases updates quickly when threats are discovered.

Large ecosystem: With over 5,000,000 .NET developers worldwide, you can leverage the large ecosystem by incorporating libraries from the NuGet package manager and the Visual Studio Marketplace. Find answers to technical challenges from the community, our MVPs, and our large support organization.

Open source: The .NET Foundation is an independent non-profit supporting the innovative, commercially friendly, open source .NET ecosystem. .NET has over 100,000 contributions from developers from over 3,700 companies outside of Microsoft.

In addition to the community and Microsoft, Technical Steering Group members, Google, JetBrains, Red Hat, Samsung and Unity are guiding the future of the .NET platform.

.NET CLR Functions

Following are the functions of the CLR.

It converts the program into native code.

- Handles Exceptions
- Provides type-safety
- Memory management
- Provides security
- Improved performance
- Language independent
- Platform independent
- Garbage collection

Provides language features such as inheritance, interfaces, and overloading for object-oriented programmings.

.NET Programming Languages

The .NET ecosystem supports various programming languages developed by Microsoft and third-party contributors. Some of the most widely used languages are:

- C# .NET: A modern, object-oriented language used for a wide range of application types.
- VB.NET: A language designed for ease of use, with syntax similar to traditional Basic.
- F# .NET: A functional-first programming language, well-suited for complex algorithms and data processing.
- C++ .NET: An extension of C++ designed for managed code applications.
- J# .NET: A language that provides .NET compatibility for Java developers.
- IronRuby, IronPython: .NET implementations of the Ruby and Python languages.
- Other Languages: JScript.NET, C Omega, ASML, and more.

Advantages of .NET Framework

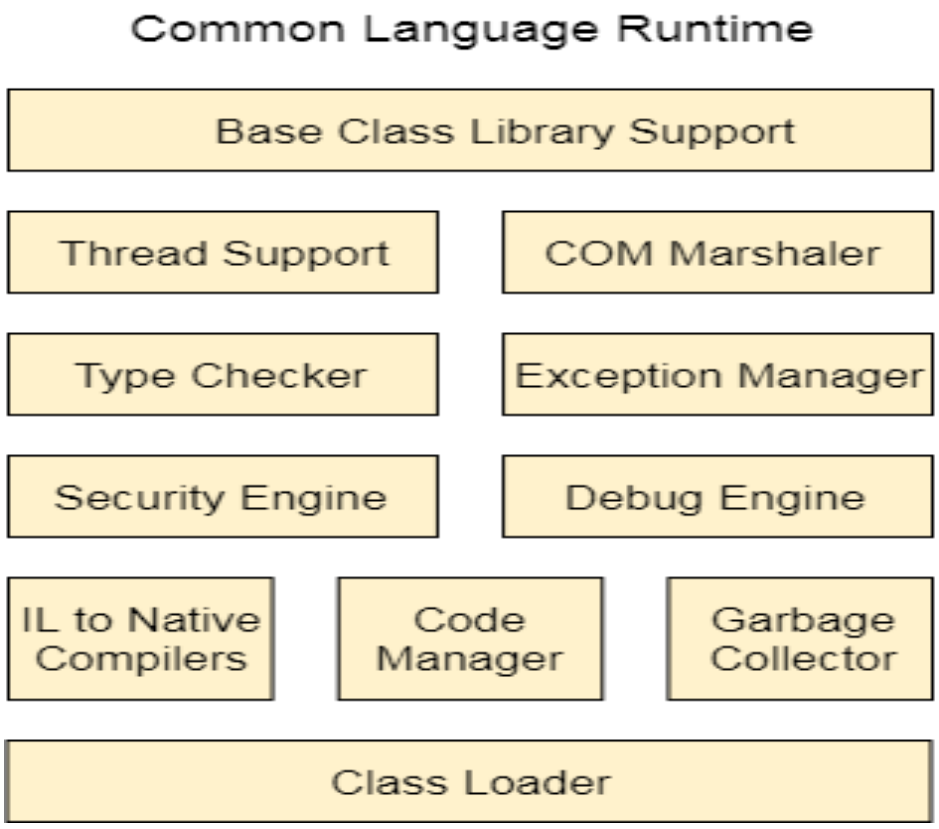
- **Multi-Language Support:** The .NET Framework supports several programming languages, including C#, F#, and Visual Basic, allowing developers to choose the language that best suits their needs while still benefiting from the same set of libraries and tools.
- **Cross-Platform Compatibility (via .NET Core and Mono):** Although initially designed for Windows, the .NET Framework can now run on various operating systems through .NET Core and Mono, making it a versatile solution for cross-platform development.
- **Large and Active Developer Community**: The .NET Framework benefits from a broad developer base, creating a rich ecosystem of libraries, tools, and resources. This helps developers solve problems more easily and share solutions across the community.
- **Security Features:** The framework includes various security features such as code access security and digital signatures, helping protect applications from malicious threats and vulnerabilities.
- **Productivity:** The set of pre-built libraries and tools offered by .NET accelerates development time, enabling developers to focus on business logic rather than reinventing common functionalities.

Disadvantages of .NET Framework

- **Windows Dependency:** Though it supports on cross-platform, the .NET Framework was originally built for Windows, and certain features may still be optimized primarily for the Windows operating system.
- **Large Footprint:** The .NET Framework installation is relatively large, which can be a concern for applications running on devices with limited storage or bandwidth.
- **Licensing Costs**: Some versions of the .NET Framework may require a paid license, adding costs to development and deployment.
- **Performance Considerations:** Although .NET provides excellent performance for most applications, it may not be the ideal choice for high-performance scenarios where low-level hardware interaction or complex algorithms are required.
- **Learning Curve:** Although .NET is designed to be easy to use, developers new to the platform may face a learning curve specially with the object-oriented programming.

.NET CLR Structure

Following is the component structure of Common Language Runtime.



- **Base Class Library Support:** It is a class library that provides support of classes to the .NET application.
- **Thread Support:** It manages the parallel execution of the multi-threaded application.
- **COM Marshaler:** It provides communication between the COM objects and the application.
- **Type Checker:** It checks types used in the application and verifies that they match to the standards provided by the CLR.
- **Code Manager:** It manages code at execution run-time.
- **Garbage Collector:** It releases the unused memory and allocates it to a new application.
- **Exception Handler:** It handles the exception at runtime to avoid application failure.
- **ClassLoader:** It is used to load all classes at run time.

What is common type system function in ASP.NET?

The common type system performs the following functions: Establishes a framework that helps enable cross-language integration, type safety, and high-performance code execution. ... Provides a library that contains the primitive data types (such as Boolean, Byte, Char, Int32, and UInt64) used in application development.

CTS support two different kinds of types:

Value Types: Contain the values that need to be stored directly on the stack or allocated inline in a structure.

Reference Types: Store a reference to the value's memory address and are allocated on the heap.

What is CTS in .Net?

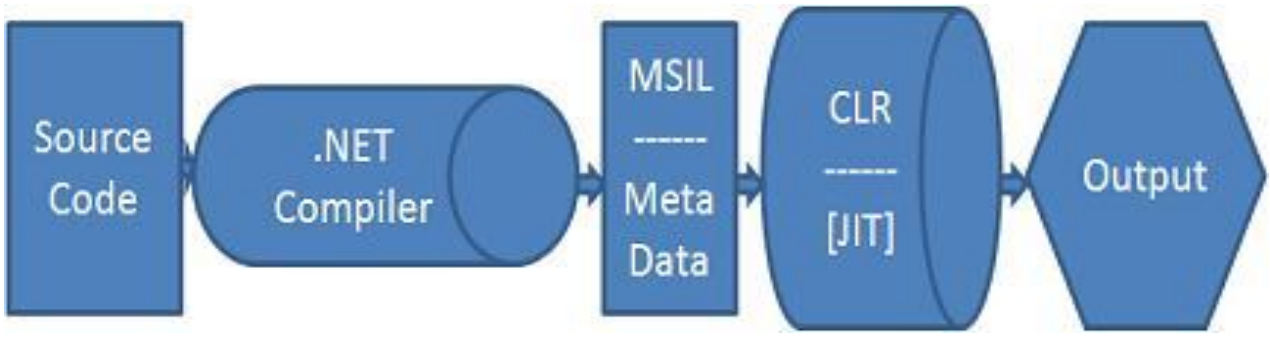
In .Net, we can write code in any language, but then IL code should be able to interact with each other, so there has to be a common way to define all supported types, that’s where type standardization is required, and CTS define that.

- Here are the few things CTS is responsible for.
- Define a set of rules for data types that all .net languages must follow
- Make sure cross-language execution possible in framework.
- Provide a library that contains the basic types used in application development like string, Byte, Char, Date etc.
- CTS defines several categories of types like Classes, Enums, Structures, Interfaces, Delegates
- Also define property types, access modifiers types, and how inheritance and overloading works etc.

- CTS component deals with data type, so that we can develop .net application in different languages, and still can communicate with each other, every language has its own data type, and one language data type can be different from other languages.

How to Microsoft .Net Framework

Microsoft .Net Languages Source Code are compiled into Microsoft Intermediate Language (MSIL) . MSIL we can call it as Intermediate Language (IL) or Common Intermediate Language (CIL). Microsoft Intermediate Language (MSIL) is a CPU independent set of instructions that can be converted to the native code. Metadata also created in the course of compile time with Microsoft Intermediate Language (MSIL) and stored it with the compiled code . Metadata is completely self-describing . Metadata is stored in a file called Manifest, and it contains information about the members, types, references and all the other data that the Common Language Runtime (CLR) needs for execution.



The Common Language Runtime (CLR) uses metadata to locate and load classes, generate native code, provide security, and execute Managed Code. Both Microsoft Intermediate Language (MSIL) and Metadata assembled together is known as Portable Executable (PE) file. Portable Executable (PE) is supposed to be portable across all 32-bit operating systems by Microsoft .Net Framework.

During the runtime the Common Language Runtime (CLR)'s Just In Time (JIT) compiler converts the Microsoft Intermediate Language (MSIL) code into native code to the Operating System. The native code is Operating System independent and this code is known as Managed Code , that is, the language's functionality is managed by the .NET Framework . The Common Language Runtime (CLR) provides various Just In Time (JIT) compilers, and each works on a different architecture depends on Operating Systems, that means the same Microsoft Intermediate Language (MSIL) can be executed on different Operating Systems. From the following section you can see how Common Language Runtime (CLR) functions.

Namespaces : A namespace is designed for providing a way to keep one set of names separate from another. The class names declared in one namespace does not conflict with the same class names declared in another.

NameSpace is the Logical group of types or we can say namespace is a container (e.g Class, Structures, Interfaces, Enumerations, Delegates etc.), example System.IO logically groups input output related features , System.Data.SqlClient is the logical group of ado.net Connectivity with Sql server related features. In Object Oriented world, many times it is possible that programmers will use the same class name, Qualifying NameSpace with class name can avoid this collision.

Example :-

```
// Namespace Declaration

using System;

// The Namespace

namespace MyNameSpace

{

   // Program start class

   class MyClass

   {

      //Functionality

   }

}
```

Namespaces allow you to create a system to organize your code. A good way to organize your namespaces is via a hierarchical system. You put the more general names at the top of the hierarchy and get more specific as you go down. This hierarchical system can be represented by nested namespaces. Bellow shows how to create a nested namespace. By placing code in different sub-namespaces, you can keep your code organized.

.NET Framework Class Library (FCL)